Medoq : Cahier des charges technique (brief développeur)

Oct 29, 2026 · @Amadou NIANG

1. Objet de la mission

Je cherche un développeur pour rendre Medoq prêt à un lancement public à Dakar : 9 chantiers prioritaires (phase 1), estimés à environ 6 semaines, sur une base de code qui fonctionne déjà.

Medoq est une plateforme qui permet aux patients de Dakar de trouver un médicament disponible dans une pharmacie proche, de le réserver (mis de côté 2 heures), de le payer par Wave, Orange Money ou en espèces, puis de le retirer ou de se le faire livrer. Les pharmacies gèrent leur stock et leurs réservations dans un espace dédié. Medoq prend une commission de 1,5 % par transaction.

Ce qui est attendu du développeur :

1. Lire ce document et l'existant (section 2), puis me renvoyer un devis et un planning pour la phase 1.
2. Réaliser la phase 1 (section 3), en respectant les règles métier (section 5) et les exigences de qualité (section 6).
3. Livrer un code testé, documenté et déployé, validé par les critères de la section 7.
4. Chiffrer séparément les phases 2 et 3 (section 4), qui feront l'objet d'une commande ultérieure.

Contact du porteur de projet : Amadou Niang, fondateur, niangamadou394@gmail.com · +33 6 17 56 63 42 (appel et WhatsApp).

2. Existant technique

Le cœur du produit est `medoq-api` : une API Node.js/Express sur PostgreSQL, hébergée sur Render, qui sert aussi l'application web (PWA). Les autres dépôts sont des prototypes ou des pages annexes.

2.1 Dépôts

Dépôt	Contenu	Technologie	État
medoq-api	API REST + application web patient/pharmacie + page d'accueil + pages légales	Node.js 22, Express 4, PostgreSQL (`pg`), JWT, bcryptjs, express-validator	En production, **base de travail**
medoq-dashboard	Tableau de bord pharmacie avancé (stock, réservations, statistiques, admin)	Angular	Prototype, utilise un service WebSocket qui n'existe pas côté API
medoq-mobile	Application patient	Flutter (seul le build web est dans le dépôt, pas le code source Dart)	Prototype
medoq-pharma	Ancienne version de l'application web	HTML/JS	Remplacé par `medoq-api/public`
carte-medoq	Carte de visite numérique	HTML	Hors périmètre

2.2 Organisation de `medoq-api`

Dossier / fichier	Rôle
`server.js`	Démarrage Express, CORS, routes `/api/v1/*` , fichiers statiques, initialisation de la base
`src/database/db.js`	Pool PostgreSQL et création du schéma (idempotent, au démarrage)
`src/database/seed.js`	Données de démonstration (si la base est vide)
`src/routes/` + `src/controllers/`	`auth` , `medications` , `pharmacies` , `reservations` , `payments` , `admin`
`src/middleware/auth.js`	`authenticate` , `requireRole` , `optionalAuth`
`src/services/sms.js` / `email.js`	Africa's Talking (OTP, notifications) / Nodemailer Gmail (alertes admin)
`scripts/onboard-pharmacy.js`	Import d'une pharmacie + stock depuis un CSV `medicament;quantite;prix`

Dossier / fichier	Rôle
`public/`	Application web (`index.html` , ~120 Ko, tout-en-un), `landing.html` , mentions légales, confidentialité, cookies, remboursement
`render.yaml`	Déploiement Render (service web + base `medoq-db` en offre gratuite)

2.3 Points d'entrée de l'API (`/api/v1`)

Module	Routes existantes
`auth`	`send-otp` , `register` , `login` , `refresh` , `logout` , `GET/PUT me` , `forgot-password` , `reset-password` , `register-pharmacy`
`medications`	`GET /` (recherche), `popular` , `categories` , `cmu` , `GET /:id` , `POST /` (admin)
`pharmacies`	`nearby?lat&lng&radius&medicationId` , `me` , `GET /` , `GET /:id` , `GET /:id/stock` , `PUT /:id/stock/:medicationId`
`reservations`	`POST /` , `GET /` (patient), `pharmacy` , `all` (admin), `GET /:id` , `/:id/cancel` , `/:id/ready` , `/:id/complete`
`payments`	`initiate` , `stats` (admin), `reservation/:id` , `/:id/confirm`
`admin`	`bootstrap` , `pharmacies/pending` , `pharmacies` , `pharmacies/:id/activate` , `pharmacies/:id/reject` , `stats`

2.4 Base de données

Tables : `users` , `otp_codes` , `medications` , `pharmacies` , `pharmacy_users` , `pharmacy_stock` , `reservations` , `payments` , `refresh_tokens` , plus la séquence `reservation_ref_seq` . Les dates sont stockées en `TEXT` ISO et les booléens en `INTEGER` 0/1 (héritage d'une ancienne version SQLite). Il n'y a pas d'outil de migration : le schéma est créé par `CREATE TABLE IF NOT EXISTS` au démarrage.

2.5 Variables d'environnement

`DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN` (86400), `JWT_REFRESH_EXPIRES_IN` (604800), `AT_API_KEY`, `AT_USERNAME`, `AT_SENDER_ID`, `OTP_REQUIRED`, `SMTP_*`, `ADMIN_EMAIL`, `ADMIN_BOOTSTRAP_SECRET`, `COMMISSION_RATE` (1.5), `RESERVATION_EXPIRY_HOURS` (2), `DELIVERY_FEE` (1500), `PORT`, `NODE_ENV`. Le modèle est dans `.env.example`. Les valeurs réelles seront transmises séparément, jamais par e-mail en clair ni dans le dépôt.

2.6 Ce qui n'existe pas encore

- Aucun test automatisé, aucune intégration continue.
- Paiement Wave / Orange Money **simulé** : `payments.controller.js` génère une URL fictive `https://pay.medoq.sn/checkout/...`.
- Pas de tâche planifiée (expiration des réservations).
- Pas de temps réel (WebSocket / SSE) vers l'espace pharmacie.

3. Travaux demandés : phase 1 (lancement)

La phase 1 regroupe 9 chantiers, tous dans `medoq-api` ; sans eux, Medoq ne peut pas accueillir de vrais patients ni encaisser de vrais paiements.

Réf.	Chantier	Priorité	Charge estimée
T1	Expiration automatique des réservations	Bloquant	2 j
T2	Paiement réel Wave et Orange Money + remboursements	Bloquant	10 j
T3	Alerte temps réel des nouvelles réservations côté pharmacie	Bloquant	3 j
T4	Pharmacies de garde gérées par l'API	Bloquant	3 j
T5	Sécurité : secret JWT, limitation des requêtes, en-têtes	Bloquant	2 j
T6	Suppression de compte et export des données	Bloquant	1,5 j
T7	Gestion du catalogue et suspension de comptes (admin)	Important	2 j
T8	Migrations de base de données et tests automatisés	Important	4 j

Réf.	Chantier	Priorité	Charge estimée
T9	Passage en hébergement payant, sauvegardes, supervision	Important	1 j

Les charges sont mes estimations ; le développeur les corrige dans son devis.

T1. Expiration automatique des réservations

Aujourd'hui une réservation `CONFIRMED` ou `READY` dont `expires_at` est dépassé reste indéfiniment active et le stock décrémenté n'est jamais rendu.

- Ajouter le statut `EXPIRED`.
- Tâche planifiée toutes les 5 minutes (cron Render ou `setInterval` dans le process, au choix du développeur, à justifier) qui, **dans une transaction** : passe en `EXPIRED` les réservations `CONFIRMED` / `READY` expirées, réincrémente `pharmacy_stock.quantity`, déclenche le remboursement si un paiement mobile est `COMPLETED` (T2).
- La tâche doit être idempotente (deux exécutions simultanées ne rendent pas le stock deux fois) : utiliser `UPDATE … WHERE status IN (…) RETURNING` ou `SELECT … FOR UPDATE SKIP LOCKED`.
- SMS au patient 30 minutes avant l'expiration, puis à l'expiration.
- Afficher le statut « Expirée » et son filtre dans l'application patient et dans l'espace pharmacie.

T2. Paiement réel Wave et Orange Money

`POST /payments/initiate` renvoie aujourd'hui une URL fictive. À remplacer par les vraies API marchandes.

- **Wave** : API Checkout (création de session, redirection, webhook de confirmation signé). **Orange Money** : API Web Payment Sonatel (ou un agrégateur comme PayDunya / CinetPay si le développeur le recommande, à argumenter dans le devis : coût, délais d'ouverture du compte marchand).
- Nouvelles routes : `POST /payments/webhooks/wave`, `POST /payments/webhooks/orange` : vérification de la signature, traitement idempotent (un même événement reçu deux fois ne crée pas deux paiements).
- Statuts de paiement : `PENDING` → `COMPLETED` | `FAILED` | `REFUNDED`. Le front interroge le statut au retour de la page de paiement.
- **Remboursement** automatique en cas d'annulation, d'expiration (T1) ou de refus par la pharmacie, conformément à `public/remboursement.html`.

- Commission : conserver le calcul existant (`COMMISSION_RATE` , 1,5 %) et l'enregistrer sur chaque paiement.
- Tout se teste d'abord en environnement *sandbox* ; les clés de production sont fournies par moi.
- Paiement `CASH` : comportement actuel conservé (validé à la remise du médicament).

T3. Alerte des nouvelles réservations côté pharmacie

La pharmacie ne sait qu'une réservation est arrivée qu'en rafraîchissant la page.

- Canal temps réel : **Server-Sent Events** (`GET /api/v1/pharmacies/me/events`) recommandé pour sa simplicité ; WebSocket accepté.
- Événements : `reservation.created` , `reservation.cancelled` , `reservation.expired` , `payment.completed` .
- Dans l'espace pharmacie : son + bandeau + compteur, sans rechargement.
- Doublon par SMS au numéro de la pharmacie si l'événement n'a pas été vu dans les 5 minutes.
- Notifications push (Web Push) : optionnel, à chiffrer à part.

T4. Pharmacies de garde

L'interface affiche déjà « Pharmacie de garde » mais l'API ne connaît pas cette notion.

- Table `on_duty_shifts` (`pharmacy_id` , `starts_at` , `ends_at`).
- Routes admin : créer, modifier, supprimer un tour de garde ; import CSV du planning hebdomadaire.
- Route publique `GET /pharmacies/on-duty?lat&lng&at=` (par défaut maintenant), triée par distance.
- Relier le bouton « Pharmacie de garde » de l'application à cette route.
- Horaires d'ouverture : stocker `opening_hours` dans un format structuré (JSON par jour) et calculer « ouvert / fermé » côté API, fuseau `Africa/Dakar` .

T5. Sécurité

- **Supprimer le secret par défaut** `'medoq-secret'` (dans `src/middleware/auth.js` et `src/controllers/auth.controller.js`) : le serveur refuse de démarrer si `JWT_SECRET` manque en production.
- Limitation des requêtes (`express-rate-limit` ou équivalent) : `send-otp` et `forgot-password` limités par numéro **et** par IP (ex. 3 SMS / 15 min par numéro), `login` limité par IP. Les SMS sont facturés : c'est une protection financière.
- Limiter le nombre d'essais de saisie d'un code OTP (5 maximum, puis code invalidé).

- Générer les OTP avec `crypto.randomInt` au lieu de `Math.random`.
- En-têtes de sécurité (`helmet`), taille maximale du corps des requêtes.
- Ne plus renvoyer `err.message` brut au client en production (route `bootstrap` notamment).
- Révoquer tous les jetons de rafraîchissement au changement de mot de passe.

T6. Données personnelles

- `DELETE /auth/me` : suppression du compte patient (anonymisation des réservations conservées pour la comptabilité : nom et téléphone effacés).
- `GET /auth/me/export` : export JSON des données du patient.
- Purge automatique des codes OTP expirés et des jetons de rafraîchissement périmés.
- Boutons correspondants dans le profil patient.
- Page CGU (`public/cgu.html`) : je fournis le texte, le développeur l'intègre et ajoute la case à cocher à l'inscription.

T7. Administration

- `PUT /medications/:id` et désactivation d'un médicament (admin).
- Suspension / réactivation d'un patient ou d'une pharmacie (`is_active`), avec effet immédiat sur les jetons en cours.
- Le refus d'une pharmacie (`/admin/pharmacies/:id/reject`) doit aussi supprimer ou désactiver le compte utilisateur du responsable, et non seulement le lien `pharmacy_users`.
- Une page d'administration simple dans `public/` (liste des pharmacies en attente, activation, statistiques, catalogue, gardes). Pas besoin d'Angular.
- Import du stock CSV depuis l'espace pharmacie (aujourd'hui seulement en ligne de commande avec `scripts/onboard-pharmacy.js`).

T8. Migrations et tests

- Introduire un outil de migration (`node-pg-migrate` ou équivalent) ; la première migration reprend le schéma actuel sans perte de données.
- Corriger les types quand c'est sûr : `TIMESTAMPTZ` pour les dates, `BOOLEAN` pour les drapeaux, `NUMERIC` pour les montants. Migration testée sur une copie de la base de production.
- Tests automatisés (Jest ou Vitest + Supertest, base PostgreSQL de test) couvrant au minimum : inscription/connexion/OTP, création de réservation concurrente sur la dernière boîte, annulation, expiration (T1), webhooks de paiement (T2), contrôles de

rôles.
- Intégration continue GitHub Actions : tests à chaque push.

T9. Hébergement et exploitation

- Passer le service web et la base Render sur une offre payante (pas de mise en veille, sauvegardes quotidiennes). Me donner le coût mensuel avant de souscrire.
- Surveillance de disponibilité sur `/health` avec alerte e-mail.
- Journal des erreurs centralisé (Sentry ou équivalent, offre gratuite).
- Environnement de *staging* séparé de la production.

4. Phases 2 et 3 : à chiffrer séparément

Ces évolutions ne font pas partie de la commande initiale ; je demande seulement une estimation pour chacune.

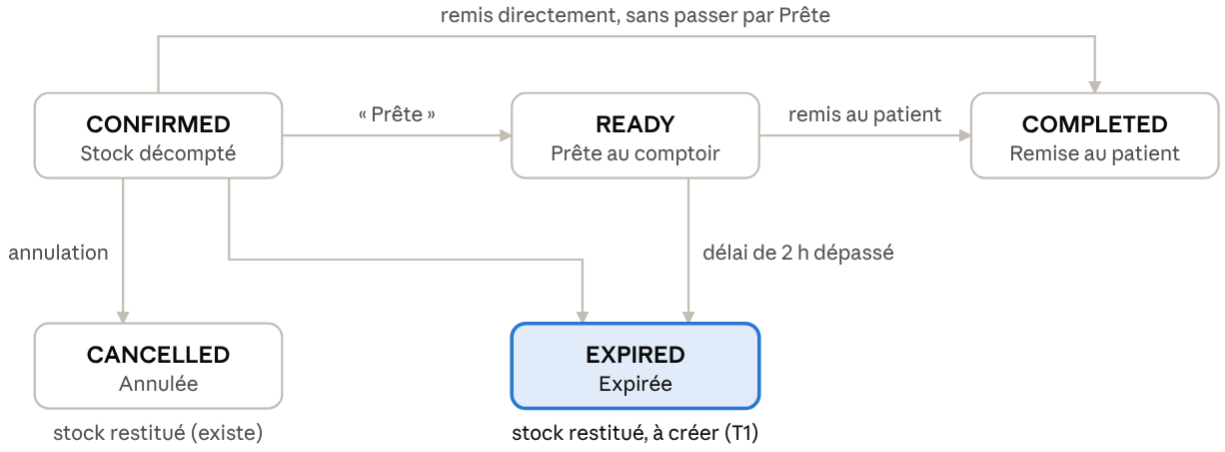
Réf.	Phase	Fonction	Notes techniques
E1	2	Panier multi-médicaments	Nouvelle table `reservation_items` ; une réservation = une pharmacie, plusieurs lignes ; un seul paiement
E2	2	Photo d'ordonnance	Upload (stockage objet type S3 / Cloudinary), validation par le pharmacien avant `READY`, accès restreint (donnée de santé)
E3	2	Plusieurs employés par pharmacie	Invitation par SMS, rôles `PHARMACY_OWNER` / `PHARMACY_STAFF`
E4	2	Médicaments équivalents	Proposer les produits de même DCI et dosage quand le stock est à zéro
E5	2	Reversements aux pharmacies	Table `payouts`, relevé mensuel (encaissé − commission), export CSV
E6	2	Reçu numérique	PDF ou page publique signée envoyée par SMS
E7	3	Avis et notes	Table `reviews`, uniquement après une réservation `COMPLETED`, recalcul de `rating`
E8	3	Carte des pharmacies	Leaflet + OpenStreetMap
E9	3	Livreurs	Rôle `COURIER`, statuts `OUT_FOR_DELIVERY` / `DELIVERED`, attribution

Réf.	Phase	Fonction	Notes techniques
E10	3	Alerte de retour en stock	Abonnement patient → SMS quand `quantity` repasse au-dessus de 0
E11	3	Application mobile publiée	Reprise du code source Flutter (absent du dépôt, à retrouver ou reconstruire), publication Play Store / App Store
E12	3	Wolof	Internationalisation de l'interface (fichiers de traduction)
E13	3	QR code de retrait	QR de la référence `MRX-…`, scanné depuis l'espace pharmacie

5. Règles métier à respecter

Ces règles sont déjà codées (sauf mention contraire) et ne doivent pas être cassées par les nouveaux développements.

Une réservation se termine toujours : remise, annulée ou expirée



cycle de vie d'une réservation · 5 statuts

Le statut `EXPIRED` est le seul à créer ; toute sortie vers `CANCELLED` ou `EXPIRED` rend le stock à la pharmacie et rembourse un paiement mobile déjà encaissé.

Règle	Valeur	Où dans le code
Durée de réservation	2 h	`RESERVATION_EXPIRY_HOURS`
Commission Medoq	1,5 % du montant	`COMMISSION_RATE`, `payments.controller.js`

Règle	Valeur	Où dans le code
Frais de livraison	1 500 FCFA, Dakar intramuros	`DELIVERY_FEE`
Validité d'un code SMS	10 min	`auth.controller.js`
Blocage après échecs de connexion	5 essais → 15 min	`auth.controller.js`
Mot de passe	6 caractères minimum	`auth.routes.js`
Rayon de recherche par défaut	10 km (distance haversine)	`pharmacies.controller.js`
Seuil d'alerte stock	5 unités par défaut	`pharmacy_stock.threshold`
Référence de réservation	`MRX-AAAA-000123`, issue d'une séquence	`makeRef()`

Autres règles :

- **Stock** : décrémenté à la réservation par un `UPDATE … WHERE quantity >= $1` atomique ; deux patients ne peuvent pas réserver la dernière boîte.
- **Médicament sur ordonnance** : livraison interdite, retrait en pharmacie, paiement sur place, le patient certifie avoir une ordonnance valide.
- **Annulation** : possible uniquement au statut `CONFIRMED`.
- **Paiement en espèces** : passe à `COMPLETED` automatiquement quand la réservation est remise.
- **Pharmacie** : invisible des patients tant qu'un administrateur ne l'a pas activée (`is_active = 1`) ; numéro de licence unique.
- **Premier administrateur** : créé une seule fois via `/admin/bootstrap` protégé par `ADMIN_BOOTSTRAP_SECRET`.
- **Mode démo** : les comptes et données de démonstration ne doivent jamais se mélanger aux vraies données.
- **Montants** en FCFA entiers, numéros au format `+221`, heure de référence `Africa/Dakar`.

6. Exigences de qualité, sécurité et conformité

Medoq manipule des données de santé et de l'argent : la qualité du code et la sécurité passent avant la vitesse de livraison.

Code

- Respecter la structure existante (`routes` → `controllers` → `database` / `services`), le style et la langue des messages (français).
- Requêtes SQL toujours paramétrées (`$1` , `$2` …), jamais de concaténation de saisies utilisateur.
- Toute opération qui touche au stock et à l'argent se fait dans une transaction.
- Aucun secret dans le dépôt (il est **public**) ; mettre à jour `.env.example` pour chaque nouvelle variable.
- Pas de régression : l'application web existante (`public/index.html`), le mode démo et la page d'accueil continuent de fonctionner.

Performance et usage

- Réponse de l'API en moins de 500 ms pour la recherche et les pharmacies proches (hors démarrage à froid).
- Application utilisable en 3G sur un téléphone Android d'entrée de gamme.

Conformité

- Loi sénégalaise n° 2008-12 sur les données personnelles (déclaration CDP gérée par moi) : minimisation des données, accès restreint aux réservations, suppression sur demande (T6).
- Les réservations révèlent des traitements médicaux : ne jamais les journaliser en clair (logs, Sentry) avec le nom ou le téléphone du patient.
- Medoq met en relation et ne vend pas de médicaments : aucun développement ne doit faire apparaître Medoq comme vendeur.

7. Critères d'acceptation

La phase 1 est acceptée quand chacun de ces tests passe sur l'environnement de staging, puis en production.

- [ ] Un patient s'inscrit avec un vrai code SMS, réserve, paie par Wave et reçoit son médicament : la réservation finit `COMPLETED` , le paiement `COMPLETED` , la commission est enregistrée.
- [ ] Même parcours avec Orange Money, puis en espèces.
- [ ] Une réservation payée puis annulée est remboursée et le stock est restitué.
- [ ] Une réservation non retirée passe `EXPIRED` dans les 5 minutes suivant l'expiration ; le stock est restitué une seule fois, même si deux tâches tournent en même temps.
- [ ] Le patient reçoit le SMS d'avertissement 30 minutes avant l'expiration.
- [ ] Deux réservations simultanées sur la dernière boîte : une seule réussit (test automatisé).

- [ ] Un webhook de paiement reçu deux fois ne crée qu'un paiement ; un webhook à signature invalide est rejeté.
- [ ] L'espace pharmacie affiche une nouvelle réservation en moins de 10 secondes, avec un son, sans rechargement.
- [ ] Les pharmacies de garde de l'heure courante s'affichent, triées par distance.
- [ ] Le serveur refuse de démarrer en production sans `JWT_SECRET`.
- [ ] Le 4e envoi de code SMS vers un même numéro en 15 minutes est refusé.
- [ ] Un patient supprime son compte : il ne peut plus se connecter, ses données personnelles sont effacées.
- [ ] Une pharmacie non validée ou suspendue n'apparaît jamais dans les résultats.
- [ ] Un médicament sur ordonnance ne peut pas être livré.
- [ ] Les tests automatisés passent dans GitHub Actions.
- [ ] Le mode démo et l'application web existante fonctionnent comme avant.

8. Livrables, modalités et réponse attendue

J'attends un devis détaillé par chantier (T1 à T9), un planning et les réponses aux questions ci-dessous.

Livrables

1. Code sur le dépôt `medoq-api`, une branche et une *pull request* par chantier, décrite et relue avant fusion.
2. Migrations de base de données et tests automatisés (T8).
3. Documentation : `README` (installation locale, variables d'environnement, déploiement), description des nouvelles routes de l'API.
4. Application déployée en staging puis en production.
5. Une session de passation d'1 h (visio) pour m'expliquer l'exploitation : paiements, gardes, sauvegardes.

Modalités

- Point d'avancement chaque semaine (WhatsApp ou visio, 30 min) et démonstration en staging à la fin de chaque chantier.
- Ordre de réalisation suggéré : T5 et T8 d'abord (sécurité et filet de tests), puis T1, T2, T3, T4, T6, T7, T9.
- Le code produit appartient à Medoq. Accord de confidentialité signé avant l'accès aux clés et à la base de production.
- Garantie : correction gratuite des anomalies découvertes pendant 30 jours après la mise en production.

- Paiement échelonné, à préciser dans le devis (par exemple 30 % à la commande, 40 % à la livraison en staging, 30 % à la mise en production).

Fournis par moi

- Accès aux dépôts GitHub et à Render.
- Comptes marchands Wave et Orange Money (ou l'agrégateur retenu), compte Africa's Talking.
- Textes juridiques (CGU) et planning des gardes de Dakar.

Questions pour le développeur

- Quel prestataire recommandez-vous pour Wave et Orange Money (en direct ou via un agrégateur), et pourquoi ?
- Server-Sent Events ou WebSocket pour T3 ?
- Faut-il garder l'application web en un seul fichier `index.html` ou la découper, et à quel coût ?
- Quel coût mensuel d'hébergement prévoyez-vous pour 1 000 réservations par mois ?
- Le code source Flutter de `medoq-mobile` est absent : êtes-vous à l'aise pour le reprendre ou le reconstruire en phase 3 ?